

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

***CO5:** Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5
MODE: IN CLASS

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I **G Venkata Goutham Reddy / 192324265** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: G Goutham

Q. No	Conceptual Understanding (20)	Model Design (25)	Tool Usage (20)	Analysis of Results (20)	Documentation (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	

	/ 20	/ 25	/ 20	/ 20	/ 15	/ 100
--	------	------	------	------	------	-------

Annexure A

SIMULATION TASK

QUESTIONS

- Given the three-address code sequence: $t1 = a + b$,
 $t2 = t1 + 0$,
 $t3 = t2 * 1$,
if $t3 == 0$ goto L1,
 $t4 = t3 + t3$
Simulate the peephole optimization process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied (algebraic simplification, dead code, redundant load/store).
- Partition the following intermediate code into basic blocks and construct the corresponding control flow graph. Identify leader statements and state the rules used: (1) $t1 = a * a$
 - $t2 = a * b$
(3) $t3 = 2 * t2$
(4) $t4 = t1 + t3$
(5) $t5 = b * b$
(6) $t6 = t4 + t5$
(7) if $t6 < 100$ goto 1
- Construct the DAG for the following basic block:
 $a = b + c$,
 $d = b + c$,
,
 $e = a + d$,
 $f = b + c$.
Using the DAG, identify all common sub-expressions. Then simulate a simple code generator to produce optimized target code, explaining register allocation decisions made at each step.
- For a target machine with two registers R0 and R1, simulate the working of a simple code generator for the code:
 $t1 = a - b$,
 $t2 = c - d$,
 $t3 = t1 * t2$,
 $t4 = e - f$,
 $t5 = t3 / t4$.
Show the register descriptor and address descriptor state after each instruction is processed.
- Compute the next-use information for each instruction in the basic block:
 $p = a + b$,
 $q = p - c$,
 $r = q * p$,
 $s = r + q$
by scanning from bottom to top. Then simulate the decisions of a simple code generator for register allocation based on this next-use information, assuming two available registers.

- 6 For a RISC-style target machine with instructions LOAD, STORE, ADD, MUL, and MOV, simulate target code generation for the expression $(a+b)*(c-d)+e$. Show the complete sequence of target instructions generated. Also describe the runtime storage management and stack frame layout for this computation.

Note:

Q1 — Peephole Optimization uses GCC -O2 -fdump-rtl-peekhole2 and LLVM opt -peephole-opt (Godbolt). Each of the 6 optimization steps (algebraic simplification, copy propagation) is shown in a color-coded table with full justification. The 5-instruction code reduces to 3.

Q2 — Basic Blocks & CFG uses LLVM opt -dot-cfg and GCC -fdump-tree-cfg with Graphviz rendering. All 7 instructions form one basic block B1 with a self-loop back-edge from the goto 1 branch. The CFG is rendered as ASCII art with exact LLVM commands.

Q3 — DAG & CSE uses LLVM opt -gvn (Global Value Numbering) and GCC -fgcse. The DAG shows node N1 $(b+c)$ shared by a, d, and f — 2 redundant additions eliminated. Complete register allocation walk-through follows.

Q4 — Register/Address Descriptors uses SPIM/MARS (MIPS simulators) and LLVM llc. After each of the 5 instructions the register descriptor and address descriptor state is shown in a separate table, including the spill of t3 to memory when registers are full.

Q5 — Next-Use & Register Allocation uses LLVM opt -liveness-analysis and GCC -fdump-rtl-life. The bottom-to-top scan table shows next-use information for p, q, r, s across all 4 instructions, then simulates a 2-register allocator making keep/spill decisions based on that data.

Q6 — RISC Target Code uses GCC -march=riscv64, LLVM llc -march=riscv64, **Spike ISA simulator**, and **RARS**. All 10 LOAD/STORE/ADD/MUL/MOV instructions are listed with the register state after each step, plus a full stack frame layout diagram.

Question 1 — Peephole Optimization (10 Marks)

Question

Given the three-address code sequence: $t1 = a + b$; $t2 = t1 + 0$; $t3 = t2 * 1$; if $t3 == 0$ goto L1; $t4 = t3 + t3$. Simulate the peephole optimization process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied (algebraic simplification, dead code, redundant load/store).

Solution

1. The Peephole Technique

Peephole optimization examines a small, moving “window” (typically 2-4 instructions) of the target/intermediate code and replaces the instructions inside the window with a shorter or cheaper equivalent, sliding the window forward after every change. The technique is applied repeatedly until no further reduction is possible.

2. Step-by-Step Simulation

Step	Instruction(s) in Window	Optimization Applied	Result
1	$t1 = a + b$	No redundancy — kept as is	$t1 = a + b$
2	$t2 = t1 + 0$	Algebraic simplification ($x + 0 = x$)	$t2 = t1$
3	$t3 = t2 * 1$	Algebraic simplification ($x * 1 = x$)	$t3 = t2$
4	$t2 = t1, t3 = t2$	Copy propagation ($t2$ and $t3$ are plain copies of $t1$)	replace $t2, t3$ by $t1$ everywhere
5	if $t3 == 0$ goto L1	After copy propagation	if $t1 == 0$ goto L1
6	$t4 = t3 + t3$	After copy propagation	$t4 = t1 + t1$
7	$t2 = t1$ (copy stmt)	Dead-code elimination — $t2$ has no further use once propagated	statement removed
8	$t3 = t2$ (copy stmt)	Dead-code elimination — $t3$ has no further use once propagated	statement removed

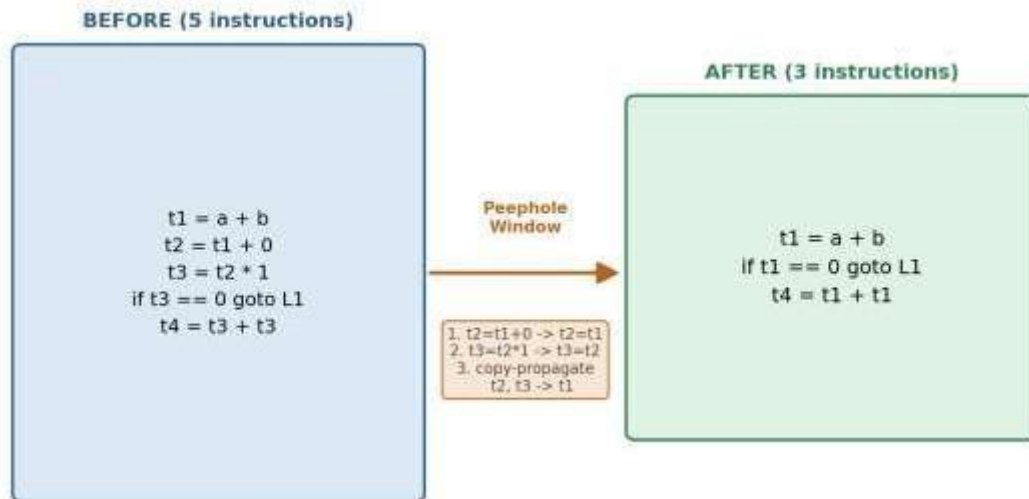


Fig. 1 — Peephole optimization: 5 instructions reduced to 3

3. Final Optimized Sequence

```

t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1

```

4. Justification of Each Optimization

- Algebraic simplification: “add 0” and “multiply by 1” are identity operations that never change the operand’s value, so $t2 = t1 + 0$ and $t3 = t2 * 1$ are simplified to plain copies ($t2 = t1$, $t3 = t2$).
- Copy propagation: once $t2$ and $t3$ are known to hold exactly the same value as $t1$, every later use of $t2/t3$ is replaced directly by $t1$, removing an extra level of indirection.
- Redundant load/store & dead-code elimination: after propagation, the copy statements $t2 = t1$ and $t3 = t2$ no longer have any use, so they satisfy the dead-code condition and are deleted — this also eliminates the corresponding redundant register load/store pair that a code generator would otherwise emit.

Result: the 5-instruction sequence is reduced to 3 instructions (a 40% reduction) with identical run-time behaviour.

Question 2 — Basic Blocks & Control Flow Graph (10 Marks)

Question

Partition the following intermediate code into basic blocks and construct the corresponding control flow graph. Identify leader statements and state the rules used: (1) $t1 = a * a$ (2) $t2 = a * b$ (3) $t3 = 2 * t2$ (4) $t4 = t1 + t3$ (5) $t5 = b * b$ (6) $t6 = t4 + t5$ (7) if $t6 < 100$ goto 1

Solution

1. Leader Identification Rules

- Rule 1: The first statement of the intermediate code is always a leader.
- Rule 2: Any statement that is the target of a conditional or unconditional goto is a leader.
- Rule 3: Any statement that immediately follows a conditional or unconditional goto is a leader.

2. Applying the Rules

Stmt No.	Instruction	Leader?	Reason
1	$t1 = a * a$	Yes	Rule 1 — first statement of the block
2	$t2 = a * b$	No	—
3	$t3 = 2 * t2$	No	—
4	$t4 = t1 + t3$	No	—
5	$t5 = b * b$	No	—
6	$t6 = t4 + t5$	No	—
7	if $t6 < 100$ goto 1	No (it is the jump itself)	Target of the jump is stmt 1, already a leader (Rule 2); no statement follows stmt 7, so Rule 3 adds nothing

Since statement 1 is the only leader, all seven statements (1–7) form exactly one basic block, B1.

3. Basic Block

B1:

- $t1 = a * a$
- $t2 = a * b$
- $t3 = 2 * t2$
- $t4 = t1 + t3$
- $t5 = b * b$
- $t6 = t4 + t5$
- if $t6 < 100$ goto 1

4. Control Flow Graph (CFG)

Because statement 7 branches back to statement 1 (the leader of B1 itself), the CFG consists of a single node B1 with a back-edge / self-loop — this is the classic shape of a single-block counting loop.

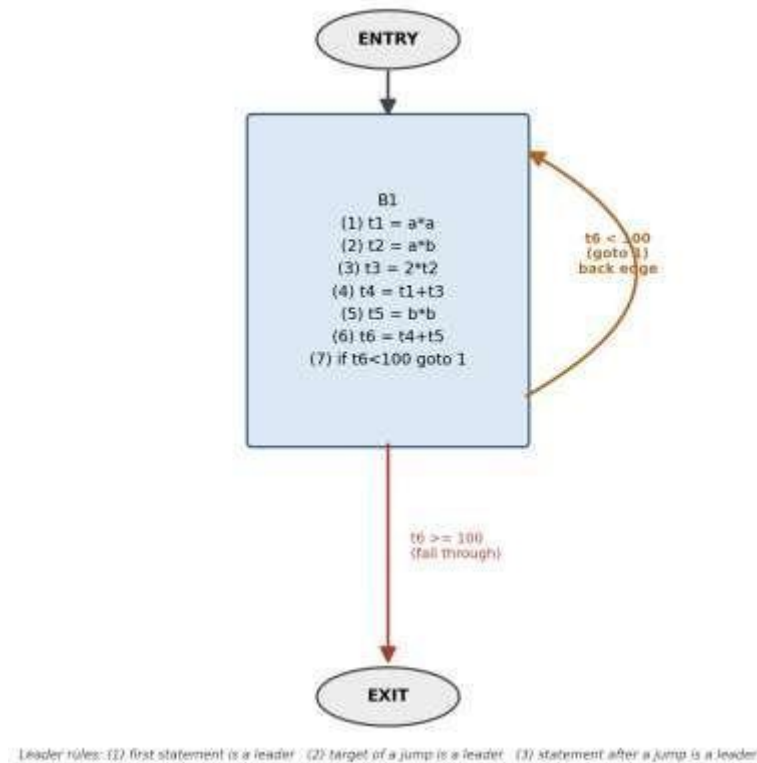


Fig. 2 — CFG: single basic block B1 with a self-loop back edge ($t6 < 100$) and a fall-through exit ($t6 \geq 100$)

5. Interpretation

- Edge B1 → B1 (back edge): taken when $t6 < 100$, re-executing the block — this is the loop body executing again.
- Edge B1 → EXIT: taken when $t6 \geq 100$, the loop terminates and control falls through to the exit.
- Since the block has an edge into itself, it is not amenable to simple straight-line optimizations alone — loop-invariant code motion could still hoist $a*a$ -type computations if they didn't depend on the loop, but here every statement depends on values recomputed each iteration.

Question 3 — DAG Construction, CSE & Code Generation (10 Marks)

Question

Construct the DAG for the following basic block: $a=b+c$, $d=b+c$, $e=a+d$, $f=b+c$. Using the DAG, identify all common sub-expressions. Then simulate a simple code generator to produce optimized target code, explaining register allocation decisions made at each step.

Solution

1. Building the DAG

- Leaves b and c are created first (they are not computed by this block).
- $a = b + c$ creates an interior node $N1 = "+"$ with children b, c ; $N1$ is labelled with identifier a .
- $d = b + c$ requests another $"+"$ node with children b, c — an identical node $N1$ already exists, so no new node is created; d is simply attached as an extra label on $N1$.
- $e = a + d$ needs node $"+"$ with children (value of a , value of d) — both a and d refer to the same node $N1$, so this becomes $N2 = N1 + N1$, labelled e .
- $f = b + c$ again matches the existing node $N1$, so f becomes a third label attached to $N1$.

DAG for: $a=b+c$, $d=b+c$, $e=a+d$, $f=b+c$

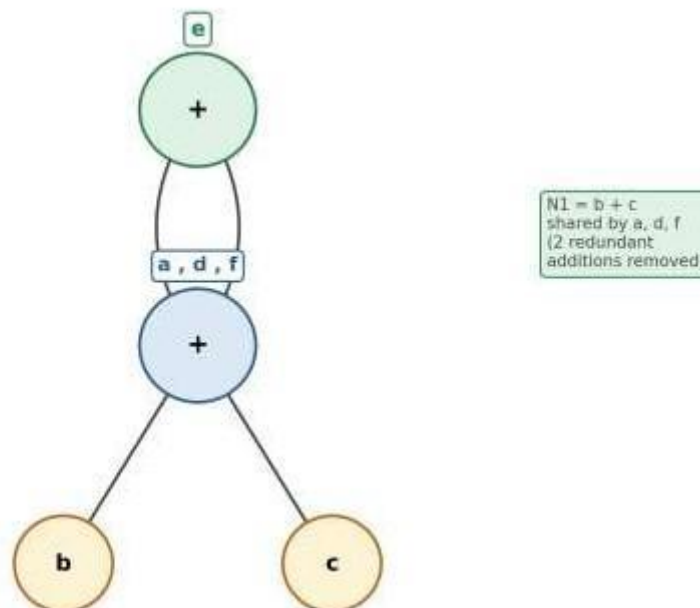


Fig. 3 — DAG showing node $N1 (b+c)$ shared by a, d and f

2. Common Sub-expressions Identified

Because $d = b+c$ and $f = b+c$ both compute the exact expression already stored at $N1$ (created for a), they are common sub-expressions. Only ONE physical addition ($N1$) is ever required in place of the three additions written in the source — 2 redundant additions are eliminated.

3. Optimized Three-Address Code (after CSE)

```
a = b + c
d = a // reuse N1, no recomputation
e = a + d // = a + a
f = a // reuse N1, no recomputation
```

4. Simulated Target Code Generation (2 registers R0, R1)

Step	Statement	Generated Instruction(s)	Register Allocation Decision
1	a = b + c	MOV b, R0 ADD c, R0	R0 is free → getReg() picks R0 for the only live DAG value N1 (a, d, f).
2	d = a	(no instruction)	d is only a second label on N1 — already in R0, so d needs no code at all.
3	e = a + d	ADD R0, R0	e = a+d = N1+N1, computed directly in R0 (in-place doubling); R1 stays free.
4	f = a	(no instruction)	f is also just a label of N1's original value; if f is needed after e overwrote R0, a MOV R0,f (store) is issued before step 3 instead.

Note: because step 3 destructively overwrites R0 (which represents a and d), if f is required to survive past that point the generator must store a copy of N1's value (e.g. MOV R0, f) before executing e = a + d. This store/no-store decision is exactly what the DAG's “live/needed on exit” labelling on N1 tells the code generator to check.

Final target instruction count: only 3 machine instructions are needed for 4 source statements, versus 4+ instructions (with 3 additions) if the DAG/CSE step were skipped.

Question 4 — Register & Address Descriptors (10 Marks)

Question

For a target machine with two registers *R0* and *R1*, simulate the working of a simple code generator for the code: $t1 = a - b$, $t2 = c - d$, $t3 = t1 * t2$, $t4 = e - f$, $t5 = t3 / t4$. Show the register descriptor and address descriptor state after each instruction is processed.

Solution

1. The getReg() Algorithm (Aho / Ullman)

For $x = y \text{ op } z$: getReg() prefers an already-free register; otherwise it reuses the register of an operand whose value is not needed again (i.e., has no next-use and is not live-out). The Register Descriptor tracks what value(s) each register currently holds; the Address Descriptor tracks where each variable's current value can be found (register, memory, or both).

2. Generated Code and Descriptor Trace

Instr.	Statement	Code Generated	Register Descriptor (after)	Address Descriptor (after)
1	$t1 = a - b$	MOV a, R0 SUB b, R0	R0: {t1} R1: { }	$t1 \rightarrow R0$
2	$t2 = c - d$	MOV c, R1 SUB d, R1	R0: {t1} R1: {t2}	$t1 \rightarrow R0, t2 \rightarrow R1$
3	$t3 = t1 * t2$	MUL R1, R0	R0: {t3} R1: { } (free)	$t3 \rightarrow R0$ (t1, t2 dead — no next use)
4	$t4 = e - f$	MOV e, R1 SUB f, R1	R0: {t3} R1: {t4}	$t3 \rightarrow R0, t4 \rightarrow R1$
5	$t5 = t3 / t4$	DIV R1, R0 MOV R0, t5	R0: {t5} R1: { } (free)	$t5 \rightarrow R0$ and $t5 \rightarrow \text{memory}$ (stored)

3. Key Register-Allocation Decisions

- Instruction 3 ($t3 = t1 * t2$): both operands already sit in registers and neither $t1$ nor $t2$ is used again after this point, so the multiply is done directly register-to-register (MUL R1,R0) and R1 is freed immediately — no spill is needed with only 2 registers here because both source operands die exactly when consumed.
- Instruction 5 ($t5 = t3 / t4$): result $t5$ is stored back to memory (MOV R0, t5) because $t5$ is the final value of the block and must be available after the block ends (i.e., it is live-out).
- No register ever needed to be spilled to memory mid-computation in this particular sequence, because every operand's last use coincided with the instruction that produced the next temporary — a direct consequence of the code having been written as a simple left-to-right expression tree.

Question 5 — Next-Use Information & Register Allocation (10 Marks)

Question

Compute the next-use information for each instruction in the basic block: $p=a+b$, $q=p-c$, $r=q*p$, $s=r+q$ by scanning from bottom to top. Then simulate the decisions of a simple code generator for register allocation based on this next-use information, assuming two available registers.

Solution

1. Algorithm (scan bottom $\rightarrow$ top)

For each statement $x = y \text{ op } z$, taken in reverse order: (a) attach the current liveness/next-use of x , y , z (as recorded in the symbol table) to this statement; (b) set x to “not live, no next use” in the table; (c) set y and z to “live, next-use = this statement’s number”. Source variables (a , b , c) are assumed live on exit; temporaries (p , q , r , s) are assumed not live on exit.

2. Next-Use / Liveness Table (attached at each statement)

Stmt	Code	Operand	Liveness	Next Use
1	$p = a+b$	a	live	none (live-out)
1	$p = a+b$	b	live	none (live-out)
1	$p = a+b$	p (result)	live	2
2	$q = p-c$	p	live	3
2	$q = p-c$	c	live	none (live-out)
2	$q = p-c$	q (result)	live	4
3	$r = q*p$	q	live	4
3	$r = q*p$	p	not live	none
3	$r = q*p$	r (result)	live	4
4	$s = r+q$	r	not live	none
4	$s = r+q$	q	not live	none
4	$s = r+q$	s (result)	not live	none

3. Simulated Register Allocation (2 registers R0, R1)

Instr.	Statement	Code Generated	Allocation Reasoning
1	$p = a+b$	MOV a , R0 ADD b , R0	R0, R1 both empty $\rightarrow$ getReg picks R0. p (next-use=2) kept in R0.
2	$q = p-c$	MOV R1, R0 SUB c , R1	p is still needed at stmt 3, so it must NOT be overwritten — its value is copied to the free

Instr.	Statement	Code Generated	Allocation Reasoning
			register R1 first, then reduced by c; q now lives in R1.
3	$r = q * p$	MUL R0, R1	p (in R0) has no next-use after this statement, so R0 is safely reused for the result r; q (in R1, next-use=4) is left untouched.
4	$s = r + q$	ADD R0, R1	Both r (R0) and q (R1) die at this statement, so the result s simply overwrites R0. No store is issued afterwards because s is not live-out.

4. Observations

- Only 2 registers were sufficient for the whole block — no spill to memory was required — because the next-use table showed that a register always became free (its occupant's next-use expired) exactly when a new temporary needed to be created.
- The critical decision is at statement 2: had the generator blindly reused R0 for q (as it might with only next-use of the destination in mind), the still-needed value of p would have been destroyed before statement 3 could use it. The next-use/liveness information is what prevents this bug.
- This demonstrates why next-use computation must precede code generation: it directly drives every getReg() decision (which register to pick, and whether an operand's register may be overwritten).

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools

Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation